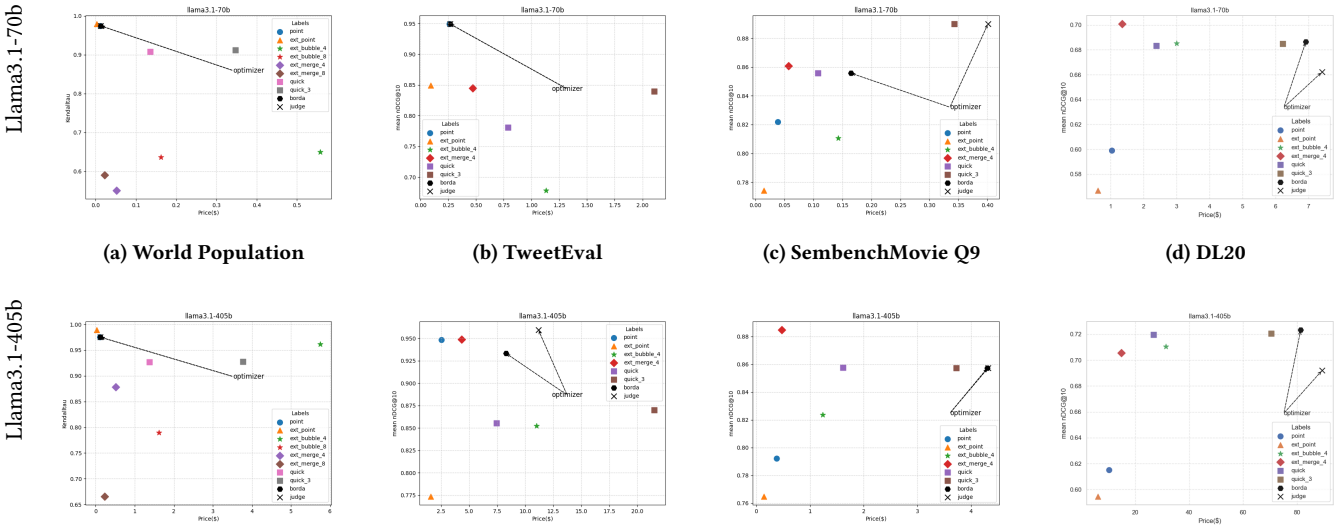


(a) World Population (b) TweetEval (c) SembenchMovie Q9 (d) DL20

(e) World Population (f) TweetEval (g) SembenchMovie Q9 (h) DL20

Figure 3: Sorting accuracy vs. the monetary budget ($).

completion token is set to 10k. Unless specified, the sample size for both membership inference stage and running candidate algorithms is fixed to 20. For the LLM-as-Judge optimizer, the Judge backbone model is always Llama3.1-405b, which can better understand the long context with stronger reasoning ability than the smaller Llama3.1-70b model [3]. For measuring the candidate algorithm quality using the derived gold ranking in the heuristic optimizer, we use kendal's tau for the world population benchmark, and *nDCG*@10 for others, reflecting their ranking objectives.

6.2 Optimizer Main Results

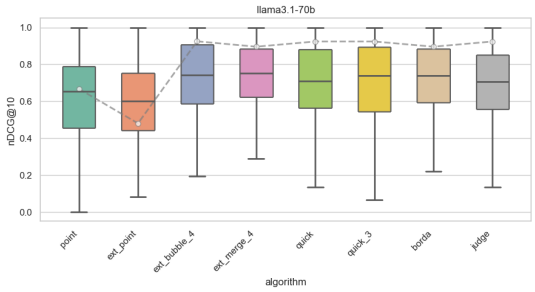
The main results of the evaluation are shown in Figure 3. The x-axis is the monetary costs and the y-axis is the ranking quality. In this set of experiments, we assume that the user does not specify a budget but the optimizer will need to decide on one algorithm and execute the algorithms on all data points.

World Population Benchmark. In the world population benchmark, our optimizer using the inquiry prompt (Prompt 4) successfully identified that the world population are factual information contained in the models' training data. For both Llama models, all 20 of the 20 evaluated samples were labeled as previously seen. As a result, the optimizer directly uses the pointwise method to achieve high accuracy: 0.974 on Llama3.1-70b and 0.975 on Llama3.1-405b. External pointwise has slightly higher accuracy, but as we have shown in Section 4, it is less stable. This observation also aligns with previous findings [10]. Compared to value-based algorithms, comparison-based approaches require a larger budget but yield lower accuracy for the factual query.

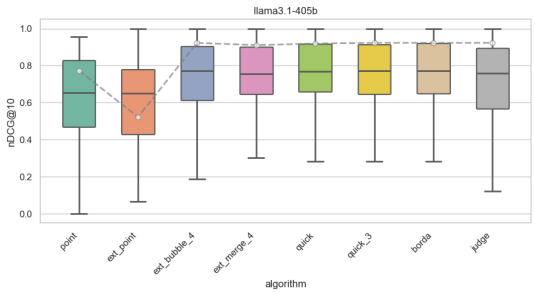
TweetEval Benchmark. The TweetEval benchmark comprises four queries and order based on four different sentiments. During the membership inference phase, for each sentiment category 20 out of 20 sampled $(tweet, category)$ pairs are identified as contained in the training data by Llama-3.1-70b. As a result, the optimizer directly uses pointwise method and achieved the highest accuracy. Using the Llama 3.1-405b model, for each of the four sentiment categories, membership inference phase identifies about 20% - 30% of the sampled $(tweet, category)$ pairs as present in the training data. Hence, pointwise method is not automatically used for these four queries. Our proposed pessimistic optimizer scores an nDCG@10 of 0.933 at a cost of $8.3, and our proposed LLM-as-Judge optimizer achieves 0.959 with a cost of $11.17. The pointwise method and our proposed external merge sort achieve nDCG@10 of 0.948. Although the pessimistic optimizer falls slightly short of the empirical upper bound established by the best single algorithm (0.948), it shows a good **robustness**, selecting the optimal algorithm in 50% of the test cases (on the irony and positivity categories). Similarly, the LLM-guided optimizer also identifies the optimal strategy in 2 out of 4 cases (irony and offensiveness). Furthermore, it achieves a higher mean nDCG@10 (0.959) than any static single-algorithm approach.

SembenchMovie Q9. The SembenchMovie Q9 benchmark consists of a single query. During the membership inference phase, approximately 25% of the movie reviews' scores are identified as previously seen by the model, indicating a mixed reliability on parametric memory. On both Llama3.1-70b and Llama3.1-405b models, the optimizer correctly gravitates toward the high-accuracy region occupied by the comparison-based algorithms. In particular, on the Llama3.1-70b model. The LLM-as-Judge strategy achieves the highest overall nDCG@10, successfully identified the best algorithm. On the Llama3.1-405b model, both the LLM-as-Judge and Borda voting strategies selected the second-best algorithm from the candidate pool. Crucially, we observe that if one were to utilize the pointwise sorting algorithm, as currently adopted in the Sembenchmark baseline [28]—ranking accuracy would degrade significantly.

DL20. The DL20 benchmark consists of 54 search queries. During the membership inference phase, we observe that only 0% – 20% of the sampled $(passage, query)$ pairs are identified as previously seen



(a) Llama3.1-70b accuracy distribution on DL20



(b) Llama3.1-405b accuracy distribution on DL20

Figure 4: Llama models' accuracy distribution on the DL20 across different algorithms and our proposed optimizers.

in the training data by both Llama 3.1 models. This low recognition rate indicates that for the vast majority of queries, the model cannot rely on memorized ground truth or parametric knowledge. Consequently, our proposed optimizer bypasses the pointwise shortcut and activates its dynamic evaluation protocols: it either employs an **LLM-guided** approach to assess the quality of algorithm outputs directly, or it derives a **Gold Ranking** (a proxy for ground truth constructed via Borda Count) to evaluate and select the optimal comparison-based sorting algorithm. As shown in Figure 3 (c) and (f), for the DL20 benchmark, the value-based approach consistently under-perform, validating our membership inference finding that these queries require reasoning rather than parametric recall.

Most notably on the Llama 3.1-405b model, our proposed **pessimistic optimizer** (labeled 'borda') demonstrates high robustness, achieving the highest nDCG@10 and effectively outperforming all static single-algorithm solutions (including quick sort and quick sort with majority voting). This indicates that the proxy gold list (using aggregated Borda Count) successfully highlights the underlying consensus ranking. We also observe that on the Llama 3.1-70b model, while the pessimistic optimizer does not surpass the **external merge sort**, it successfully achieves higher accuracy than all other single-algorithm solutions.

Conversely, the **LLM-as-Judge** optimizer (labeled 'judge'), while competitive, falls short on this dataset. We attribute this to the **long-context limitation**: unlike the TweetEval tasks which involve short text, DL20 involves re-ranking long passages. When the LLM Judge is tasked with evaluating multiple full-length permutations simultaneously, the excessive context window size likely induces "lost-in-the-middle" phenomena or attention degradation [35], rendering the Judge's decision less reliable than the mathematical consensus of the pessimistic strategy.

To further comprehend the optimizer performance on the challenging DL20 benchmark, we analyze the accuracy distributions presented in the box plots of Figure 4. For the Llama 3.1-70b model, the pessimistic optimizer achieves a median nDCG@10 comparable to external merge sort, the best-performing single algorithm, but exhibits slightly higher variance, as indicated by the extended lower whisker. However, on the more capable Llama 3.1-405b model, the pessimistic optimizer demonstrates superior stability, combining a high median score with notably low variance. This trend suggests that the efficacy of Borda Count aggregation is positively correlated with model capability: as the underlying model generates stronger candidate rankings, the consensus signal becomes more robust. In contrast, although the LLM-as-Judge optimizer maintains a competitive median score, it exhibits higher variance across both models, as the judge's accuracy distribution is significantly wider than that of the pessimistic approach.

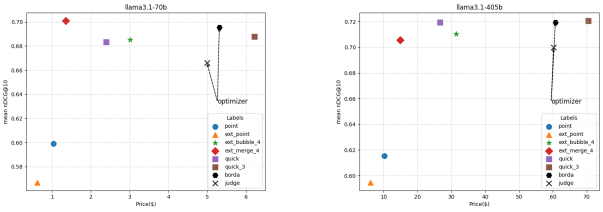
6.3 Cost Estimation

Model	Algorithm	Est($)	True($)	Diff ($)
Llama3.1-70b	point	.0107	.0109	-.0002
	ext point	.008	.003	.005
	quick	.113	.127	-.014
	quick 3	.320	.34	-.02
	ext bubble 4	.587	.557	.03
	ext merge 4	.037	.052	-.015
Llama3.1-405b	point	.103	.105	-.002
	ext point	.095	.027	.068
	quick	1.22	1.37	-.15
	quick 3	3.67	3.77	-.1
	ext bubble 4	6.06	5.75	.31
	ext merge 4	.422	.513	-.091

Table 2: Algorithms cost estimation with 20 samples. on World Population dataset

To ensure that the optimizer respects user-defined budget constraints, it is critical to accurately estimate the monetary cost of each candidate algorithm. Table 2 presents a comparison between our pre-execution cost estimates and the actual costs incurred on the World Population dataset using 20 samples. Across both Llama3.1 models, our cost model demonstrates high accuracy. For value-based algorithms, the estimation error is negligible. For comparison-based algorithms, the discrepancy also remains minimal, with the estimated cost consistently tracking the true cost within a small margin of error (e.g., a difference of only $0.15 for quick sort on the Llama3.1-405b model). We note that the relative error is largely inconsequential, whereas the absolute error is critical. If an algorithm is underestimated by a large margin and the optimizer incorrectly chosen this prohibitively expensive strategy, this can lead to a direct violation of the user's budget constraint.

6.4 With Budget Constraints



(a) $6 Budget **(b) $65 Budget**

Figure 5: Limiting the optimizer budget to $6 and $70.

To evaluate the adaptability of the optimizer to financial constraints, we imposed strict budget limits of $6 for the Llama3.1-70b and $65 for the Llama 3.1-405b. These specific thresholds were chosen to test the system's ability to filter out high-cost strategies, specifically quick sort with majority voting algorithm, which requires $6.22 on the Llama3.1-70b and $70.40 on the Llama3.1-405b. As illustrated in Figure 5, the optimizer successfully navigated these constraints. In both cases, it correctly uses budget less than the constraint. Furthermore, the pessimistic optimizer is table to acheive high accuracy. This demonstrates the optimizer's capability to perform effective cost-aware pruning of the search space while maintaining high ranking quality.

Model	Samples	Borda	Judge	Oracle
70b	15	.685 ± .008	.655 ± .016	.683 ± .003
	20	.688 ± .003	.668 ± .008	.683 ± .005
	25	.692 ± .005	.670 ± .002	.683 ± .004
	30	.690 ± .008	.661 ± .015	.686 ± .010
405b	15	.719 ± .004	.679 ± .007	.702 ± .011
	20	.720 ± .003	.688 ± .012	.697 ± .011
	25	.714 ± .007	.694 ± .009	.702 ± .009
	30	.711 ± .006	.691 ± .007	.700 ± .007

Table 3: Sample Size on DL20 with no budget constraint

6.5 Sample Size Sensitivity

Determining the optimal sample size for execution-based optimization is a non-trivial challenge. In Table 3, using DL20, we evaluate the ranking accuracy (nDCG@10) across varying sample sizes for our proposed optimizers, Borda Count aggregation (Pessimistic) and LLM-as-Judge (Optimistic), alongside with 'Oracle' approach. These three approaches use the same set of samples and we report the mean and standard deviation with three runs. The oracle approach represents the performance achievable if the optimizer had access to ground truth relevance labels to select the best algorithm based on the sample execution. We expected the oracle approach to serve as an empirical upper bound.

Interestingly, we observe that for all sample sizes and models, our proposed consensus aggregation with Borda Count consistently achieves the highest mean nDCG@10 with minimal standard deviation. This counter-intuitive result suggests that algorithm's execution on small random samples may introduce stochastic noise or outliers that diverge from the full dataset's distribution, causing the Oracle to overfit to the specific sample rather than selecting the algorithm that generalizes best to the entire corpus. In contrast, the Borda Count aggregates relative preferences across all candidate algorithms, effectively identifying a proxy ranking list that generalizes more robustly to the full dataset. The LLM-as-Judge approach provides a strong baseline, yet falls short of the consensus aggregation and Oracle, likely due to the inherent challenge of determining the quality of rankings from long contexts.

6.6 Summary

The optimistic LLM-as-Judge strategy excels at selecting the optimal algorithm for short-text tasks, leveraging its reasoning capabilities to identify fine-grained distinctions. However, on the challenging DL20 benchmark involving long passages, the Judge exhibits significant performance variance, likely due to context window limitations and information overload. The pessimistic Borda Count aggregation strategy demonstrates superior robustness. Although voting mechanisms may obscure the peak performance of a single optimal candidate through averaging effects, they concurrently mitigate the impact of low-quality outliers and hence select a more stable and robust candidate. Moreover, this aggregation strategy scales positively with model capability; on the Llama 3.1-405b model, the Borda count aggregation optimizer achieves a high mean nDCG@10 score on DL20, and outperforms any single static algorithm.

7 DISCUSSION

In this work, we presented the LLM ORDER BY semantic operator and systematically studied its physical implementations. We introduced several new algorithmic designs to improve existing semantic sorting algorithms and introduce the new external-merge semantic sort. Through a unified experimental framework across diverse sorting tasks, we demonstrated that no single access path is universally optimal. Instead, the effectiveness of an implementation depends strongly on the query characteristics and the underlying data, highlighting the importance of adaptive access-path selection in practice. We also empirically observe a test-time scaling phenomena on the semantic LLM ORDER BY operator between compute cost and ordering quality, offering a principled heuristic rule in choosing the external comparison-based candidate algorithms.

While existing systems often rely on static execution strategies for semantic Order By, these static approaches fail to adapt to different query tasks. To address this limitation, we introduce a dynamic optimizer that selects near-optimal physical implementations on the fly. Using heuristic rules, consensus aggregation, and LLM judgment, our approach successfully navigates the trade-off between accuracy and cost, consistently achieving performance superior to or on par with the best fixed single algorithm approach, thereby establishing a new standard for efficient LLM-based sorting. In future work, we plan to expand the capability of our optimizer to support other operators, such as semantic `JOIN` [59, 60], `Extract` [33, 61], and `Filter` [56].